Lab 07
Arrays, Lists and Exception Handling

Objectives: At the end of this lab session, the student should be able to:

- Use arrays in Java.
- Writing exception handling
- Use Java's Lists and ArrayLists

Part 1: Problem Solving Exercises (Time: 60 minutes)

CREATING AN ARRAY

Creation of an array involves three steps:

1. Declare the array
2. Create memory locations
3. Put values into the memory locations.

Declaration of Arrays:

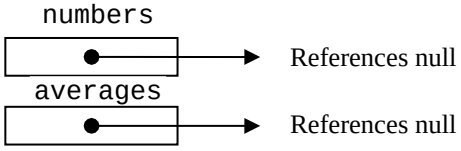
Arrays in Java may be declared in either of two forms:

Form1: `type arrayname[];`

Form2: `type [] arrayname;`

Examples:

```
int numbers[];
float averages[];
```



Further examples:

```
int [] counters;
float [] marks;
```

Remember: ***We do not enter the size of the array in the declaration.***

Creation of Arrays

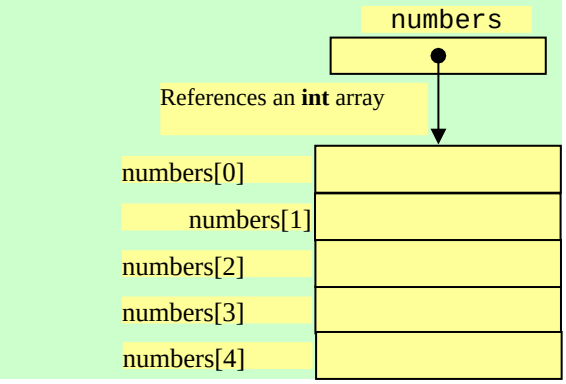
After declaring an array, we need to create it in the memory. Java allows us to create arrays using the '**new'** operator, as shown below:

```
arrayname = new type[size];
```

Examples:

```
numbers   = new int[5];
averages  = new float[10];
```

- The lines create necessary memory locations for the arrays **numbers** and **averages** and designate them as **int** and **float** respectively.
- The variable numbers refers to an array of 5 integers and averages refers to an array of 10 floating point values.
- It is also possible to combine the two steps – declaration and creation – into one as shown below:




```
int numbers[] = new int[5];
```

We can then assign values to the individual elements of the array:

```
numbers[0] = 35;
numbers[1] = 40;
.....
numbers[4] = 19;
```

Initialization of Arrays

We can initialize arrays automatically as they are declared:

```
type arrayname[] = {list of values};
```

Example: `int numbers[] = {35, 40, 20, 57, 19};`

It is also possible to assign an array object to another: For example:

```
int a[] = {1, 2, 3};     int b[];       b = a;
```

are valid in Java. Both arrays will have the same values (they reference the same memory location).

Array Length

In Java, all arrays store the allocated size in a public field called **length**. We can access the length of an array **x** using **x.length**. Example:

- ***int elements=x.length;***

Exercise 1: Using Arrays:

1.1 Create a new Java project and add to it the following class:

```
import java.util.Scanner;
public class ArrayApp {
        public static void main(String[] args) {
                int readings[] = new int[5];
                Scanner input = new Scanner(System.in);
                for (int i = 0; i < readings.length; i++) {
                        System.out.print("Enter element no. " + i + " -->");
                        readings[i] = input.nextInt();
                }
        input.close();
        }
}
```

1.2 Add a linear search method that searches for a value in an integer array. The method returns the index of the first found value. If the value is not found, then the method returns -1. Callers of this method should check the return value. Use the following signature:

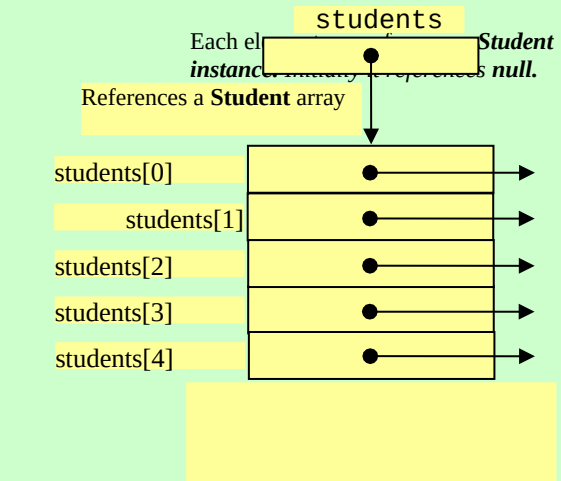
```
public static int search(int theArray[], int value)
```

1.3 Add the method *replace ()* that searches for a value in an integer array, and if found, then that value is replaced by another value. Let the method replace all occurrence of the *searchValue* with the *replaceValue*. Use the following signature:

```
public static void replace(int theArray[], int searchValue, int replaceValue)
```

1.4 Call the replace method in your main() method to replace every occurrence of the value 2 with the value 9, and then print the array elements to see if they were changed.

Exercise 2: Arrays of Objects:




```
Student students[] = new Student[5];
```

This creates an array of 5 elements that can hold references to Student instances, but the instances are not yet created. We call a constructor for each Student and store the object reference in the individual elements of the array:

```
students[0] = new Student("Ahmed");
students[1] = new Student("Ali");
              ......
students[4] = new Student("Zakia");
```

Perform the following:

2.1 Open the Circle project created in ***Exercise1-Lab05***.

2.2 Create a new class and call it *CircleArrayTest*, and copy the following code

```
import java.util.Scanner;
public class CircleArrayTest {
	public static void main(String arg[]) {
		Scanner input = new Scanner(System.in);
		System.out.println("Enter the size of the array:");
		int size = input.nextInt();
		Circle circles[] = new Circle[size];
		for (int i = 0; i < circles.length; i++) {
			System.out.println("Enter the radius");
			double radius = input.nextDouble();

			circles[i]=new Circle(radius);
		}
		System.out.println("Circles List:");
		//write code to print the radius, area and circumference
		//for all the circles, each on a line.
		input.close();
	}
}
```

2.3 Write code to print the radius, area and circumference for all the *circles*, each on a line.

2.4 In the CircletArrayTest class, add a *static* method called *findLargest (Circle circles[])* that returns the *Circle* that has the "largest" *radius* in the array parameter (*assuming that only one circle satisfies this request*). Use an *enhanced loop* (aka *foreach* loop).

2.5 In the main method write a statement to print all the details of the circle that has the largest radius.

Exception Handling

In the above main method, if the user enters a string for the radius an exception called "*InputMismatchException*" will be thrown. Use *try/catch* to handle that exception by displaying an error message and assigning zero values to the radius, so your program will not terminate and continue to read data of the next Circle.

Exercise 3: ArrayLists:

Unlike fixed-size arrays, `ArrayLists` stretch as you add elements to them and shrink as you remove elements from them. The declaration: ArrayList<**T**> *list*;
declares *list* as an ArrayList collection that can store only objects of type **T** where <T> is a placeholder for any non-primitive type. The actual ArrayList is created using the new operator:

ArrayList<**T**> *list* = new ArrayList<T> ();

Now, *list* is created but has no elements. Its size is zero, and will grow as we add elements to it.
The following table shows some of the methods that you can use with an ArrayList:

Method	Description
`add`	Adds an element to the end of the ArrayList.
`clear`	Removes all the elements from the ArrayList.
`contains`	Returns true if the ArrayList contains the specified element; otherwise,returns false.
`get`	Returns the element at the specified index.
`indexOf`	Returns the index of the first occurrence of the specified element in the ArrayList.
`remove`	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
`size`	Returns the number of elements stored in the ArrayList.
`trimToSize`	Trims the capacity of the ArrayList to current number of elements.

Perform the following:

- Create a new project and then copy the following class to the new project and then run it:

```
import java.util.ArrayList;
public class ArrayListCollection {
   public static void main( String[] args )   {
      // create a new ArrayList of Strings with an initial capacity of 10
      ArrayList< String > items = new ArrayList< String >();

      items.add( "red" ); // append an item to the list
      items.add( 0, "yellow" ); // insert the value at index 0

      // header
      System.out.print(
         "Display list contents with counter-controlled loop:" );

      // display the colors in the list
      for ( int i = 0; i < items.size(); i++ )
         System.out.printf( " %s", items.get( i ) );

      // display colors using foreach in the display method
      display( items,
         "\nDisplay list contents with enhanced for statement:" );

      items.add( "green" ); // add "green" to the end of the list
      items.add( "yellow" ); // add "yellow" to the end of the list
      display( items, "List with two new elements:" );

      items.remove( "yellow" ); // remove the first "yellow"
      display( items, "Remove first instance of yellow:" );

      items.remove( 1 ); // remove item at index 1
      display( items, "Remove second list element (green):" );

      // check if a value is in the List
      System.out.printf( "\"red\" is %sin the list\n",
         items.contains( "red" ) ? "": "not " );

      // display number of elements in the List
```

```
         System.out.printf( "Size: %s\n", items.size() );
      } // end main

      // display the ArrayList's elements on the console
      public static void display( ArrayList< String > items, String header ) {
         System.out.print( header ); // display header

         // display each element in items
         for ( String item : items )
            System.out.printf( " %s", item );

         System.out.println(); // display end of line
      } // end method display
   } // end class ArrayListCollection
   /**************************************************************************
    * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
    * Pearson Education, Inc. All Rights Reserved.                           *
    *************************************************************************/
```

- Add a static method *getRepeatCount(List<String> string, String value).* Let the method return how many times that string *value* exists in the list or return 0 otherwise. Test the method by finding how many RED's are in the items list of the main method.

Part 2: Practice Exercises (Time: 90 minutes)

Exercise 4: Perform the following:

- In the CircleArrayTest class, add a static method called *getAverageArea (Circle circles[])* that returns the *average area* of all circles. Use the enhanced for loop.
- In the main method get and print the average circle area.
- Add an instance method called *enlarge()* that takes a double *scaleFactor* as parameter and if it is greater than 1, it multiplies the *radius* of the circle by this *scaleFactor*. If the *scaleFactor* is less than or equal to 1 it does nothing.
- Add a static method called `scaleUp(Circle circles[])` that enlarges each circle in the circles array using a random number between 2 and 9 inclusive.
- Use the above methods to *scaleUp* all the circles in your CircleArrayTest. Print the average area after the update.

Additional Practice Exercises:

Exercise 5: List of GradeBooks:

In this exercise, developing a GradeBook class that instructors can use to maintain students' grades on an exam and display a grade report that includes the grades, class average, lowest grade, and highest grade. So, create a package called event to store the following classes. A class called GradeBook that has two fields: a course title, and grades for exam in an array, including the setters, getters and the fully parameterized constructor.
The following is GradeBookTest, after implementing GradeBook, GradeBookTest must running correctly. You need to implement missing part in GradeBookTest.

```
public class GradeBookTest {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
		int[] grade151 = {90,50,70,80,100,90,50,70,80,95,90,55,75,85,100};
		GradeBook gradeBook1 = new GradeBook("Programming Concepts", grade151);
		display(gradeBook1);
		System.out.println("Minimum Grades: "+gradeBook1.minimum());
		System.out.println("Maximum Grades: "+gradeBook1.maximum());
		System.out.println("Average Grades: "+gradeBook1.average());

	}
	/* Write method to display course title and all grade for this course*/
	/* Write method for Array list of GradeBooks to find minimum in the GradeBooks.*/
	/* Write method for Array list of GradeBooks to find maximum in the GradeBooks.*/
	/* Write method for Array list of GradeBooks to find average in the GradeBooks.*/
}
```

Exercise 6: List of Circles:

- In Exercise2, the Circle project, re-implement ***CircleArrayTest*** in ***CircleListTest*** to use ***List*** and ***ArrayList*** instead of ***Array***.
- In the main method, calling getAverageArea may return ArithmeticException (if the list is empty). Use *try/catch* to handle this exception by displaying an error message so your program will not terminate.